

IT Infrastructure & DevOps Trainee: Practical Implementation Assignment

Task 1: System Provisioning & Linux Administration

```
sameer@sameer:~$ sudo adduser trainee
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
Changing the user information for trainee
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] Y
sameer@sameer:~$ sudo usermod -aG sudo trainee
sameer@sameer:~$ sudo -l -U trainee
User trainee may run the following commands on sameer:
    (ALL : ALL) ALL
sameer@sameer:~$ getent group sudo
sudo:x:27:sameer,trainee
sameer@sameer:~$
```

Creating user “trainee” and adding to the sudo group.

```
trainee@sameer: ~ — sudo vim /etc/ssh/sshd_config
#
# Note that the above implies that configuration options for
# /etc/ssh/sshd_config.d/*.conf take precedence over those
# file. In addition, configuration snippet files are processed
# order, so options defined in files with names that sort earlier
# precedence.
#
# Such configuration snippets may be present in default installations
# Ubuntu.
Include /etc/ssh/sshd_config.d/*.conf

# When systemd socket activation is used (the default), the
# configuration must be re-generated after changing Port,
# ListenAddress.
#
# For changes to take effect, run:
#
#   systemctl daemon-reload
#   systemctl restart ssh.socket
#
Port 2222
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
```

The above screenshot shows that configuration changes default port no of ssh (22) to 2222 and does not permit root login and password based authentication.

```

sameer@sameer:~/ssh$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/sameer/.ssh/id_rsa):
Enter passphrase for "/home/sameer/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/sameer/.ssh/id_rsa
Your public key has been saved in /home/sameer/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:l0B+HCw3W3NRsTv55rgV4AYMEi7erYAYziY7rIShnRY sameer@sameer
The key's randomart image is:
+---[RSA 4096]---+
|      .oo.   .+o|
|      .oo.*.o . .|
| .   . .ooo* + . |
|o o o o .o..o . o|
|o=E. o .S.o  o = |
|*+ o . . .   + |
|=o+  .       + |
|oo           = |
|.           o..|
+-----[SHA256]-----+
sameer@sameer:~/ssh$ ls
authorized_keys  id_rsa  id_rsa.pub

```

The above screenshot shows the ssh key has been generated. To have a successful connection generated key should exist in **/home/trainee/.ssh** path.

```

trainee@sameer:~/ssh$ sudo ss -tulnp | grep ssh
tcp  LISTEN 0      128          0.0.0.0:2222      0.0.0.0:*
    users:((("sshd",pid=8733,fd=6))
tcp  LISTEN 0      128          [::]:2222        [::]:*
    users:((("sshd",pid=8733,fd=7))
trainee@sameer:~/ssh$ ip a | grep ens33
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    inet 192.168.1.5/24 brd 192.168.1.255 scope global dynamic noprefixroute ens
33

```

The above screenshot is to verify IP address and port number of ssh in my ubuntu machine.

```

Connection to 192.168.1.5 closed.
PS C:\Users\Sameer Tamang\Desktop> ssh -p 2222 -i id_rsa root@192.168.1.5
root@192.168.1.5: Permission denied (publickey).
PS C:\Users\Sameer Tamang\Desktop> ssh -p 2222 -i id_rsa trainee@192.168.1.5
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-31-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

2 updates can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Sat Sep 12 22:29:37 2026 from 192.168.1.4
trainee@sameer:~$

```

The above screenshot verifies unsuccessful ssh connection as root user but has a successful connection as user trainee.

```

trainee@sameer:~$ sudo ufw status verbose
[sudo: authenticate] Password:
Status: inactive
trainee@sameer:~$ sudo ufw allow 2222/tcp
Rules updated
Rules updated (v6)
trainee@sameer:~$ sudo ufw allow 80/tcp
Rules updated
Rules updated (v6)
trainee@sameer:~$ sudo ufw allow 443/tcp
Rules updated
Rules updated (v6)
trainee@sameer:~$ sudo ufw enable
Firewall is active and enabled on system startup
trainee@sameer:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
2222/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
2222/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)

```

The screenshot shows that local firewall (ufw) status was inactive at first. Port numbers such as 2222, 80 and 443 are enabled and status is active.

Task 2: Containerization & Web Services

Nginx:

```
trainee@sameer:~/Task2/nginx$ ls
Dockerfile  nginx.conf
trainee@sameer:~/Task2/nginx$ cat Dockerfile
FROM nginx:alpine

COPY nginx.conf /etc/nginx/nginx.conf

trainee@sameer:~/Task2/nginx$ cat nginx.conf
events {}

http {
    server {
        listen 80;

        location / {
            proxy_pass http://backend:5000;
        }
    }
}
```

The Dockerfile creates an Nginx container and copies the **nginx.conf** file into it.

The nginx.conf tells Nginx to listen on port 80 and forward incoming requests to the Flask backend on port 5000.

Backend:

```
trainee@sameer:~/Task2/app$ cat Dockerfile
FROM python:3.12-slim

WORKDIR /app

RUN pip install --no-cache-dir flask

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]

trainee@sameer:~/Task2/app$ cat app.py
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello TechKraft, I am interested in becoming one of krafteders."

app.run(host="0.0.0.0", port=5000)
```

The Dockerfile creates a Python container with Flask, while **app.py** runs a simple web application on port 5000, allowing Nginx to forward requests to it.

Web Application:

```
trainee@sameer:~/Task2/app$ curl http://localhost:80
Hello TechKraft, I am interested in becoming one of krafteders.trainee@sameer:~/Task2/app$
trainee@sameer:~/Task2/app$ curl http://localhost:5000
Hello TechKraft, I am interested in becoming one of krafteders.trainee@sameer:~/Task2/app$ |
```

Verified the deployed web application by accessing both Nginx on port 80 and the backend application on port 5000 using **curl**. Both endpoints successfully returned the expected application response.

Docker File:

```
trainee@sameer:~/Task2$ cat docker-compose.yml
services:

  nginx:
    build: ./nginx
    ports:
      - "80:80"
    depends_on:
      - backend

  backend:
    build: ./app
    ports:
      - "5000:5000"
    depends_on:
      - database

  database:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: myapp
      MYSQL_USER: myuser
      MYSQL_PASSWORD: mypassword
      MYSQL_ROOT_PASSWORD: rootpassword
    volumes:
      - dbdata:/var/lib/mysql

volumes:
  dbdata:
```

This **docker-compose.yml** runs three services together: Nginx on port 80, the Flask backend on port 5000, and a MySQL 8.0 database with the **dbdata** volume to keep database data persistent;

Persistent Volume:

```
trainee@sameer:~/Task2$ sudo docker exec -it task2-database-1 mysql -u myuser -pmypassword -e "SHOW DATABASES;"
[sudo: authenticate] Password:
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| Database |
+-----+
| information_schema |
| myapp           |
| performance_schema |
+-----+

trainee@sameer:~/Task2$ sudo docker exec -it task2-database-1 mysql -u myuser -pmypassword myapp -e "CREATE TABLE test (id INT);"
mysql: [Warning] Using a password on the command line interface can be insecure.
trainee@sameer:~/Task2$ sudo docker restart task2-database-1
task2-database-1
trainee@sameer:~/Task2$ sudo docker exec -it task2-database-1 mysql -u myuser -pmypassword myapp -e "SHOW TABLES;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| Tables_in_myapp |
+-----+
| test             |
+-----+
trainee@sameer:~/Task2$
```

The **myapp** database and **test** table remained available after restarting the MySQL container, confirming that the **dbdata** persistent volume is successfully preserving the database data.

Task 3: Automation & Shell Scripting

```

trainee@sameer:/opt/scripts$ cat infra_health_check.sh
#!/bin/bash

LOG="/var/log/infra_health.log"
TIME=$(date '+%Y-%m-%d %H:%M:%S')
WARNING=0

echo "=====" | tee -a "$LOG"
echo "Infrastructure Health Check - $TIME" | tee -a "$LOG"
echo "=====" | tee -a "$LOG"

# CPU, RAM and Root Disk usage
CPU=$(top -bn1 | awk '/Cpu\(s\) / {print 100 - $8}')
RAM=$(free | awk '/Mem:/ {printf "%.0f", $3/$2 * 100}')
DISK=$(df / | awk 'NR==2 {print $5}' | tr -d '%')

echo "CPU: ${CPU}% | RAM: ${RAM}% | Root Disk: ${DISK}%" | tee -a "$LOG"

# Check Backend container
BACKEND_STATUS=$(sudo docker inspect -f '{{.State.Status}}' task2-backend-1 2>/dev/null)

if [ "$BACKEND_STATUS" = "running" ]; then
    echo "Backend App: RUNNING" | tee -a "$LOG"
else
    echo "[WARNING] Backend App is ${BACKEND_STATUS:-NOT FOUND}!" | tee -a "$LOG"
    WARNING=1
fi

# Check Root Disk usage
if [ "$DISK" -gt 85 ]; then
    echo "[WARNING] Root Disk usage is ${DISK}%" | tee -a "$LOG"
    WARNING=1
else
    echo "Disk Status: OK" | tee -a "$LOG"
fi

echo "-----" | tee -a "$LOG"

# Overall status
if [ "$WARNING" -eq 0 ]; then
    echo "STATUS: OK - Infrastructure is healthy." | tee -a "$LOG"
else
    echo "STATUS: WARNING - Check the issues above!" | tee -a "$LOG"
fi

echo "=====" | tee -a "$LOG"

```

Above scripts checks RAM, CPU Disk Usage, Docker status and backend container status. Logs are stored in /var/log/infra_health.log file in a proper timestand.

```

trainee@sameer:/opt/scripts$ sudo ./infra_health_check.sh
=====
Infrastructure Health Check - 2026-09-13 00:18:21
=====
CPU: 2.4% | RAM: 51% | Root Disk: 48%
Docker: RUNNING
Backend App: RUNNING
Disk Status: OK
-----
STATUS: OK - Infrastructure is healthy.
=====

```

The above screenshot shows the result of the executed script.

Cronjob:

```
#
# m h dom mon dow   command
*/15 * * * * /opt/scripts/infra_health_check.sh
```

This cronjob runs the script every 15 minutes and stores logs.

Log Output:

```
Infrastructure Health Check - 2026-09-13 17:15:01
=====
CPU: 53.3% | RAM: 29% | Root Disk: 55%
Docker: RUNNING
Backend App: RUNNING
Disk Status: OK
-----
STATUS: OK - Infrastructure is healthy.
=====

Infrastructure Health Check - 2026-09-13 17:30:01
=====
CPU: 16.7% | RAM: 29% | Root Disk: 55%
Docker: RUNNING
Backend App: RUNNING
Disk Status: OK
-----
STATUS: OK - Infrastructure is healthy.
=====

Infrastructure Health Check - 2026-09-13 17:45:01
=====
CPU: 42.9% | RAM: 29% | Root Disk: 55%
Docker: RUNNING
Backend App: RUNNING
Disk Status: OK
-----
STATUS: OK - Infrastructure is healthy.
=====
trainee@sameer:~$
```

The above screenshot is the output of Infra health output. We can clearly see that log being generated every 15 minutes.

```
trainee@sameer:/opt/scripts$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
17e4b755d434   netdata/netdata:latest              "/usr/sbin/run.sh"      17 hours ago   Up 7 hours   0.0.0.0:19999->19999/tcp, [::]:19999->19999/tcp   netdata
7573fbd33baf   prom/prometheus:latest              "/bin/prometheus --c..." 17 hours ago   Up 7 hours   0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp       prometheus
c09e1c68934c   task2-nginx                          "/docker-entrypoint.s..." 19 hours ago   Up 7 hours   0.0.0.0:80->80/tcp, [::]:80->80/tcp              task2-nginx-1
53074eb5ac85   task2-backend                        "python app.py"         19 hours ago   Up 38 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp      task2-backend-1
24cba250ec67   mysql:8.0                           "docker-entrypoint.s..." 19 hours ago   Up 7 hours   3306/tcp, 33060/tcp                        task2-database-1

trainee@sameer:/opt/scripts$ docker stop task2-backend-1
task2-backend-1
trainee@sameer:/opt/scripts$ sudo ./infra_health_check.sh
=====
Infrastructure Health Check - 2026-09-13 18:08:33
=====
CPU: 2.4% | RAM: 28% | Root Disk: 55%
Docker: RUNNING
[WARNING] Backend App is exited!
Disk Status: OK
-----
STATUS: WARNING - Check the issues above!
=====
trainee@sameer:/opt/scripts$
```

If we stop the backend application, logs will be generated likewise as shown in above screenshot.

Metrics/Monitoring

```
trainee@sameer:~/monitoring$ cat docker-compose.yml
services:

  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
    depends_on:
      - netdata
    restart: unless-stopped
```

```
netdata:
  image: netdata/netdata:latest
  container_name: netdata
  ports:
    - "19999:19999"
  volumes:
    - netdata_config:/etc/netdata
    - netdata_lib:/var/lib/netdata
    - netdata_cache:/var/cache/netdata
    - /proc:/host/proc:ro
    - /sys:/host/sys:ro
    - /var/run/docker.sock:/var/run/docker.sock:ro
  restart: unless-stopped

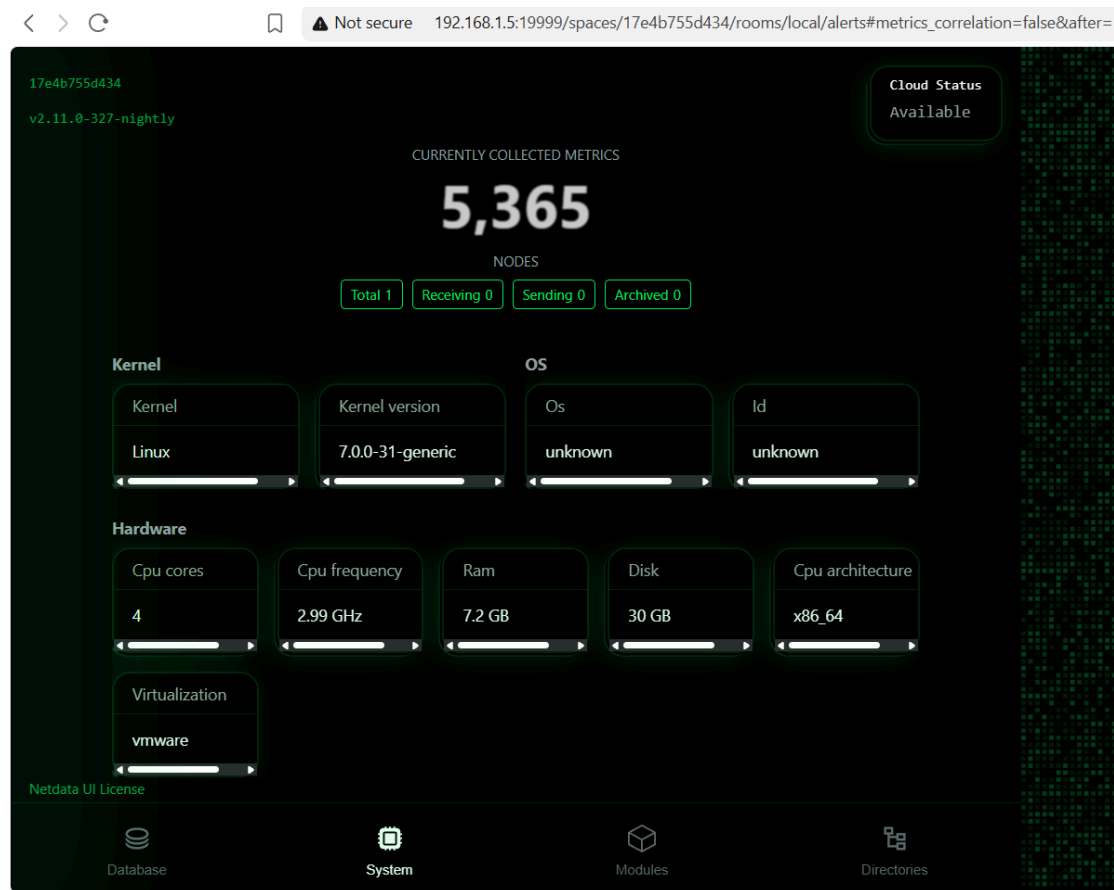
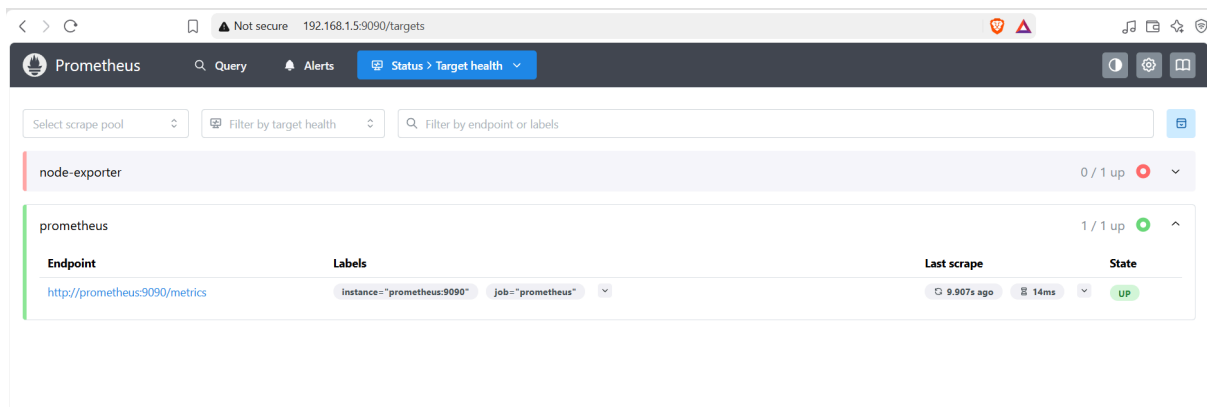
volumes:
  prometheus_data:
  netdata_config:
  netdata_lib:
  netdata_cache:
```

This Docker Compose file runs Prometheus on port 9090 and Netdata on port 19999 to monitor system and Docker container metrics.

```
trainee@sameer:~/monitoring$ cat prometheus/prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "netdata"
    static_configs:
      - targets: ["netdata:19999"]
```

This Prometheus configuration tells Prometheus to collect metrics from the Netdata container every 15 seconds using **netdata:19999** as the monitoring target.



Above screenshots displays two web dashboards hosted on a local IP address: Prometheus showing target health status with **node-exporter** down as i had removed node exporter, and Netdata monitoring system metrics, kernel info, and collected metrics.